

第四课：C语言基础以及流水灯的实现

4.1 二进制、十进制和十六进制

4.2 C语言变量类型和范围

4.3 C语言基本运算符

4.4 for循环语句

4.5 while循环语句

4.6 函数的简单介绍

4.7 软件延时

4.8 流水灯实现

4.1 二进制、十进制和十六进制

■ 十进制	二进制	十六进制
➤ 0	0b0	0x00
➤ 1	0b1	0x01
➤ 2	0b10	0x02
➤ 3	0b11	0x03
➤ 4	0b100	0x04
➤		
➤ 9	0b1001	0x09
➤ 10	0b1010	0x0A
➤ 11	0b1011	0x0B
➤ 12	0b1100	0x0C
➤ 13	0b1101	0x0D
➤ 14	0b1110	0x0E
➤ 15	0b1111	0x0F
➤ 16	0b10000	0x10
➤ 17	0b10001	0x11
➤		

每一位有几个状态

4.2 C语言的变量类型及范围

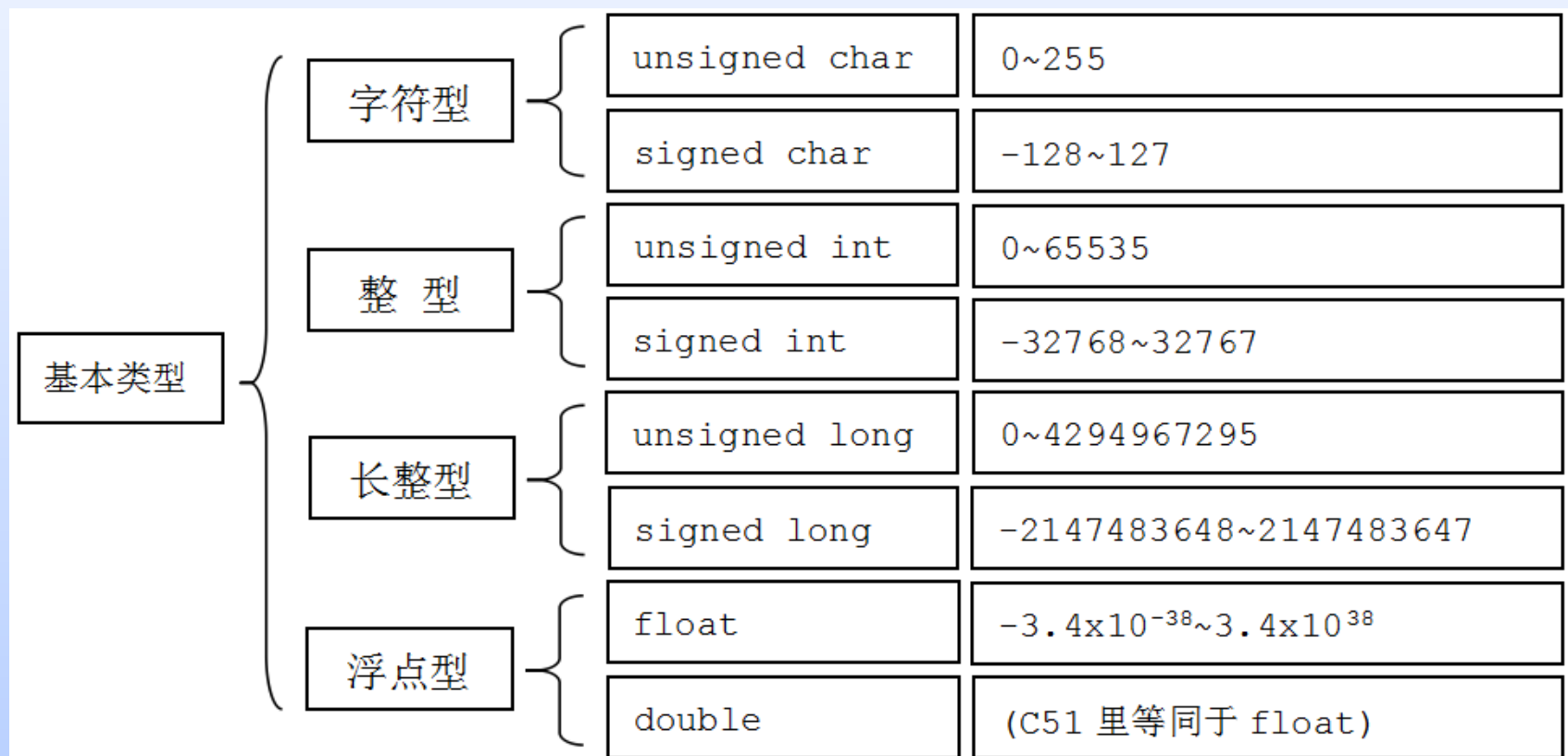
变量

变量是指在程序运行过程中或数学运算中其值可以被修改、变化的量。它相当于一个“容器”，用于存储临时数据，方便在不同场景下调用或更新。

常量

常量是指在程序运行过程中或数学运算中其值固定不变的量。它通常用于表示固定的参数、规则或已知条件。

4.2 C语言的变量类型及范围



类型	中文名称	占用内存	有效数字位数	数值范围 (约)
float	单精度浮点数	4 字节	6-7 位	$\pm 3.4 \times 10^{-38}$ 至 $\pm 3.4 \times 10^{38}$
double	双精度浮点数	8 字节	15-17 位	$\pm 1.7 \times 10^{-308}$ 至 $\pm 1.7 \times 10^{308}$

思考：如果将unsigned int i 修改为 unsigned char i，会发生什么情况？

```
1 #include<reg52.h>
2
3 sbit LED = P0^0;
4 sbit ADDR0 = P1^0;
5 sbit ADDR1 = P1^1;
6 sbit ADDR2 = P1^2;
7 sbit ADDR3 = P1^3;
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }
```

如果将i定义为unsigned char类型变量，范围在0-255内，当循环超过255溢出，i=0，

在for循环中，i始终小于30000，在for循环中始终停留。

小灯始终点亮。

编程宗旨：能用小的不用大的，但要保证逻辑正确。

4.3 C语言基本运算符

- 加、减、乘、除和取余数

- + - * / %

- a+b; a-b; a*b; a/b; a%b;

- 自加和自减

- a++ b--

- a=a+1 b=b-1

区分a++和++a:

a=0;b=0;

a=b++;//先运算后++, 先将b结果赋值给a, b再++, 结果: a=0,b=1;

a=++b;//先++后赋值, 先b++, 再将结果赋值给a, 结果: b=1,a=1;

4.3 C语言基本运算符

区分：赋值运算符= 和比较运算符== !=

=将右边的值赋给左边的变量

a=1;a=1+2;//结果： a=3。

==判断左右两边的值是否相等

if(a==1); //判断a是否为1，返回True 或 False

!=判断左右两边的值是否不相等

if(a!=1); //判断a是否不为1，返回True 或 False

4.4 for语句的用法

- **for**语句的一般形式(用法1)

```
for(表达式1;表达式2;表达式3)
{
    (需要执行的语句);
}
```

用法2:

```
for(i=0;i<30000;i++);
```

用法3:

```
for(;;)
```

用法1与用法2区别在于**for**语句后面的分号：用法1没有分号，分号表示结束，因为还要进行打括号中的操作；用法2的分号实际相当于是空操作，在不断执行的过程中耗时，延时。

用法3相当于**while(1)**死循环，习惯上更常见的用法是**while(1)**。

4.5 while语句的用法

- while语句的一般形式

- while(表达式)

```
{  
    循环语句;  
}
```

```
while (1)  
{  
    LED = 0;  
    for(i=0;i<30000;i++);  
    LED = 1;  
    for(i=0;i<30000;i++);  
}
```

```
while (1)  
    LED = 0;  
    for(i=0;i<30000;i++);  
    LED = 1;  
    for(i=0;i<30000;i++);
```

4.6 函数的简单介绍

■ 函数定义的一般形式:

- 函数值类型 函数名(形式参数列表)
- {
- 函数体
- }

- 1、函数值类型
- 2、函数名
- 3、形式参数列表
- 4、函数体
- 5、main函数(必须有且唯一)
- 6、语句注释//

```
1 #include<reg52.h>
2
3 sbit LED = P0^0;
4 sbit ADDR0 = P1^0;
5 sbit ADDR1 = P1^1;
6 sbit ADDR2 = P1^2;
7 sbit ADDR3 = P1^3;
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }
```

1, 有返回值还是没有返回值 return x; 如果x是unsinged char, 那么函数值类型就是 unsigned char, 如果没有返回值, 就是void。

2, 函数名可以任意字母、数字和下划线组成, 但是数字不能作为开头。
函数名不能与其他函数或变量重名, 也不能是关键字。

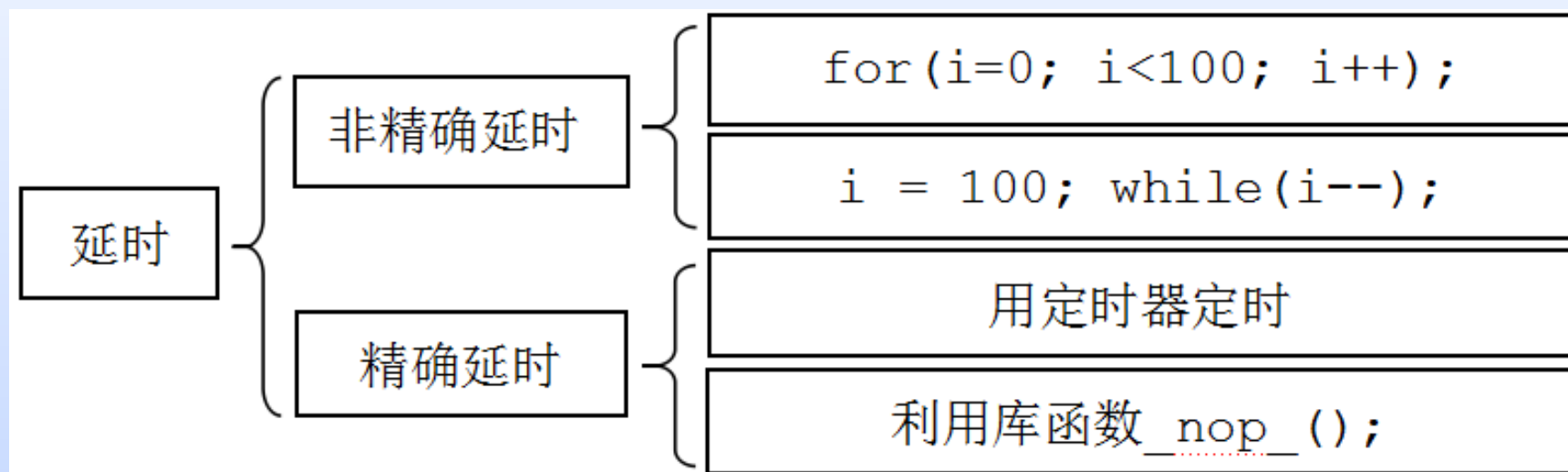
3, 形参列表, 函数调用时, 传递数据用; 如果函数不需要传递参数, 可以用void, void可以省略。但是包含形参的括号不能省略。

4, 函数体, 包括声明部分和执行语句部分。声明必须要在执行语句之前。

5, 有且仅有一个main, 程序执行都是从main函数开始执行的。

6, 注释是一个习惯, 友好维护协作。

4.7 51单片机延时常用办法



- **Nop**是汇编里面的用法，这节课我们先来看非精确延时。
- 生活上的常识问题：人眼分辨率，对闪烁高于**50HZ**的信号我们看到的就是常量的。如果低于**20ms**，看到就是常亮。

4.7 51单片机延时常用办法

Options for Target 'Target 1'

Device Target Output Listing User C51 A51 BL51 Locate BL51 Misc Debug Utilities

Intel 80/87C52

Xtal (MHz): 11.0592 ☐ Use On-chip ROM (0x0-0x1FFF)

Memory Model: Small: variables in DATA

Code Rom Size: Large: 64K program

Operating system: None

Off-chip Code memory

	Start:	Size:
Eprom		
Eprom		
Eprom		

Off-chip Xdata memory

	Start:	Size:
Ram		
Ram		
Ram		

☐ Code Banking

	Start:	End:
Banks: 2	Bank Area: 0x0000	0xFFFF

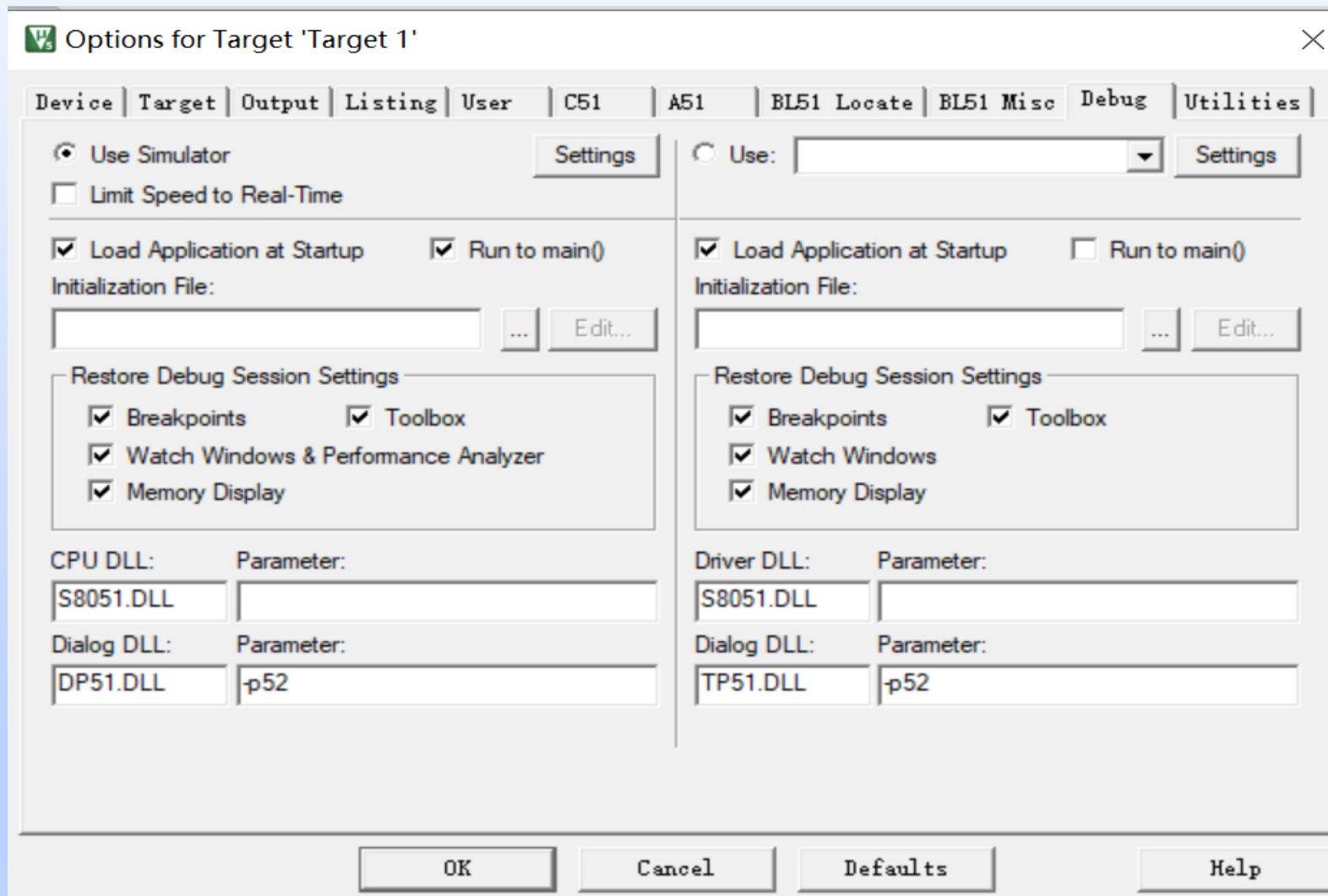
☐ 'far' memory type support

☐ Save address extension SFR in interrupts

OK Cancel Defaults Help

软件中的魔术棒-target-晶振改为与板子相同11.0592

4.7 51单片机延时常用办法



软件中的魔术棒-debug-use simulator使用模拟器点击ok
点击菜单栏debug-start/stop debug session或者点击D放大镜，出现汇编页面

4.7 51单片机延时常用办法

D:\code\lesson3\lesson3\lesson3.uvproj - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help



Registers

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x000
PC \$	C:0x0
states	389
sec	0.000
psw	0x00

```
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }
```

Disassembly

```
19:         while(1)
20:         {
21:             LED = 0;
22:             for(i=0;i<30000;i++);
23:             LED = 1;
24:             for(i=0;i<30000;i++);
25:         }
26: }
```

0x000D	C280	CLR	LED(0x80.0)
0x000F	E4	CLR	A
0x0010	FE	MOV	R6,A
0x0011	FF	MOV	R7,A
0x0012	0F	INC	R7
0x0013	BF0001	CJNE	R7,#0x00,C:0017
0x0016	0E	INC	R6
0x0017	BE75F8	CJNE	R6,#0x75,C:0012
0x001A	BF30F5	CJNE	R7,#0x30,C:0012
0x001D	D280	SETB	LED(0x80.0)
0x001F	E4	CLR	A
0x0020	FE	MOV	R6,A
0x0021	FF	MOV	R7,A
0x0022	C3	CLR	C
0x0023	EF	MOV	A,R7

4.7 51单片机延时常用办法

黄箭头表示程序运行的位置。第一个RST；第二个全速运行；第三个STOP停止。左侧有RES可以查看寄存器状态；SYS系统中有一个sec，表示程序运行的时间，单位是s。

D:\code\lesson3\lesson3\lesson3.uvproj - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

RST

Register Value

Register	Value
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x000
PC	0x000
states	389
sec	0.000
psw	0x00

```
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }
```

Disassembly

```
19:         while(1)
20:         {
21:             LED = 0;
22:             for(i=0;i<30000;i++);
23:             LED = 1;
24:             for(i=0;i<30000;i++);
25:         }
26: }
```

0x000D C280 CLR LED(0x80.0)

0x000F E4 CLR A

0x0010 FE MOV R6,A

0x0011 FF MOV R7,A

0x0012 0F INC R7

0x0013 BF0001 CJNE R7,#0x00,C:0017

0x0016 0E INC R6

0x0017 BE75F8 CJNE R6,#0x75,C:0012

0x001A BF30F5 CJNE R7,#0x30,C:0012

23: LED = 1;

0x001D D280 SETB LED(0x80.0)

24: for(i=0;i<30000;i++);

0x001F E4 CLR A

0x0020 FE MOV R6,A

0x0021 FF MOV R7,A

0x0022 C3 CLR C

0x0023 EF MOV A,R7

4.7 51单片机延时常用办法

先复位，然后在函数LED=0和LED=1这两行前分别设置断点。

如果断点设置不了，需要在魔术棒C51选项卡修改code optimization优化等级level设置为低等级0，任何语句都不优化，所有语句可以设置断点。重新设置后需要重新编译。

D:\code\lesson3\lesson3\lesson3.uvproj - μVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help



Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x000
PC \$	C:0x0
states	389
sec	0.000
psw	0x00

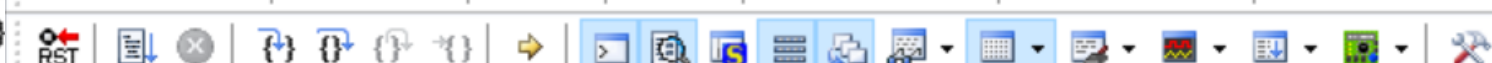
```
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }
```

Disassembly			
19:		while(1)	
20:		{	
21:		LED = 0;	
:0x000D	C280	CLR	LED(0x80.0)
22:		for(i=0;i<30000;i++);	
:0x000F	E4	CLR	A
:0x0010	FE	MOV	R6,A
:0x0011	FF	MOV	R7,A
:0x0012	0F	INC	R7
:0x0013	BF0001	CJNE	R7,#0x00,C:0017
:0x0016	0E	INC	R6
:0x0017	BE75F8	CJNE	R6,#0x75,C:0012
:0x001A	BF30F5	CJNE	R7,#0x30,C:0012
23:		LED = 1;	
:0x001D	D280	SETB	LED(0x80.0)
24:		for(i=0;i<30000;i++);	
:0x001F	E4	CLR	A
:0x0020	FE	MOV	R6,A
:0x0021	FF	MOV	R7,A
:0x0022	C3	CLR	C
:0x0023	EF	MOV	A,R7



Registers

Register
Regs
r0
r1
r2
r3
r4
r5
r6
r7
Sys
a
b
sp
sp_max
dptr
PC \$
stater
sec
psw



Registers

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x75
r7	0x30
Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x001D
stater	150613
sec	0.16342556
psw	0x00

lesson3.c STARTUP.A51

```

8 sbit ENLED = P1^4
9
10 void main()
11 {
12     unsigned int
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0; i
23         LED = 1;
24         for(i=0; i
25     }
26 }

```

- 复位后全速运行到断点1停止时间：0.00042752s
- 再点击全速运行到断点2停止时间：0.16342556s
- 两者相减163ms，可以清楚看到小灯闪烁，是否大于20ms。如果不够20ms，那么延时时间肯定不够。
- 如果优化等级和晶振设置不同，时间有差异。

4.7 51单片机延时常用办法

View –watch window –watch1 添加我们想要观察的变量，比如i,查看i当前的值。16进制7530换算10进制就是30000.

我们还可以看寄存器的值，添加一个P0,0xFE最低为是低电平。当我们想要观察寄存器，变量，可以在这里灵活去观察和获取。

Watch 1		
Name	Value	Type
 i	0x7530	uint
<Enter expression>		

计算器

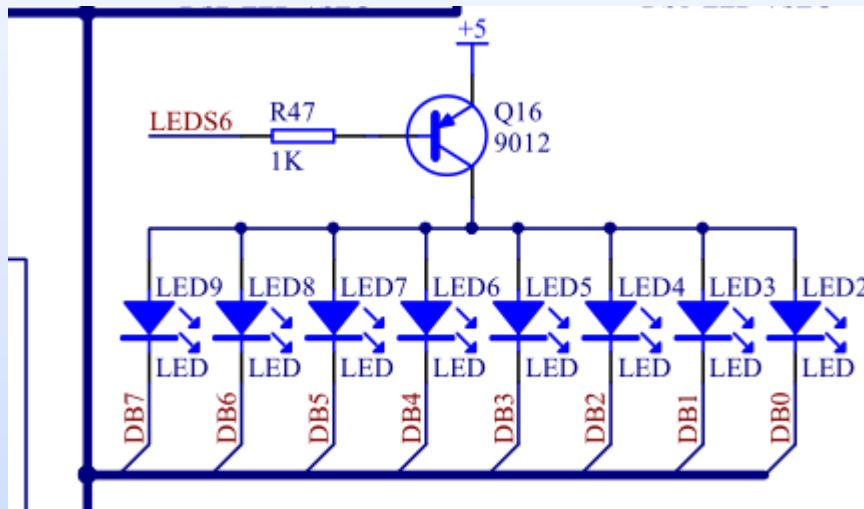
程序员

7530 =
30,000

HEX 7530
DEC 30,000
OCT 72 460
BIN 0111 0101 0011 0000

Watch 1		
Name	Value	Type
 i	0x7530	uint
 P0	0xFE '?'	uchar

4.8 流水灯实验



P0.0/AD0	39	DB 0	1
P0.1/AD1	38	DB 1	2
P0.2/AD2	37	DB 2	3
P0.3/AD3	36	DB 3	4
P0.4/AD4	35	DB 4	5
P0.5/AD5	34	DB 5	6
P0.6/AD6	33	DB 6	7
P0.7/AD7	32	DB 7	8

J6
CON8

- 8个LED发光二极管，分别对应单片机IO口的P0.0到P0.7口，8个单片机IO口组成一个字节，在程序编写过程中，可以直接用P0来进行操作。
- C语言的8位二进制数代表了8个IO口

4.8 流水灯实验

■ **P0 = 0xFE; 0b1111 1110**

➤ **0xFE, 0xFD, 0xFB, 0xF7,**

➤ **0xEF, 0xDF, 0xBF, 0x7F.**

■ **P0.7 P0.6 P0.5 P0.4 P0.3 P0.2 P0.1 P0.0**

➤ 1 1 1 1 1 1 1 0

➤ 1 1 1 1 1 1 0 1

➤ ...

```

1 #include<reg52.h>
2
3 sbit LED = P0^0;
4 sbit ADDR0 = P1^0;
5 sbit ADDR1 = P1^1;
6 sbit ADDR2 = P1^2;
7 sbit ADDR3 = P1^3;
8 sbit ENLED = P1^4;
9
10 void main()
11 {
12     unsigned int i = 0;
13
14     ENLED = 0;
15     ADDR3 = 1;
16     ADDR2 = 1;
17     ADDR1 = 1;
18     ADDR0 = 0;
19     while(1)
20     {
21         LED = 0;
22         for(i=0;i<30000;i++);
23         LED = 1;
24         for(i=0;i<30000;i++);
25     }
26 }

```

```

1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 void main()
10 {
11     unsigned int i =0;
12     ENLED = 0;
13     ADDR3 = 1;
14     ADDR2 = 1;
15     ADDR1 = 1;
16     ADDR0 = 0;
17     while(1)
18     {
19         P0=0xFE;
20         for(i=0;i<30000;i++);
21         P0=0xFD;
22         for(i=0;i<30000;i++);
23         P0=0xFB;
24         for(i=0;i<30000;i++);
25         P0=0xF7;
26         for(i=0;i<30000;i++);
27         P0=0xEF;
28         for(i=0;i<30000;i++);
29         P0=0xDF;
30         for(i=0;i<30000;i++);
31         P0=0xBF;
32         for(i=0;i<30000;i++);
33         P0=0x7F;
34         for(i=0;i<30000;i++);
35     }

```

C语言运算符

- 左移 << 右移 >>
- $X \ll 2$ $X \gg 2$
- 左移，最低位填0补充；右移，最高位填0补充
 - 0xf0 左移1位： 11100000
 - 右移1位： 01111000
- 按位取反符号 ~
 - 取反后1变成0，0变成1

0x0F取反后成为 0xF0,即00001111->11110000

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 void main()
10 {
11     unsigned int i =0;
12     unsigned char cnt = 0;
13     ENLED = 0;
14     ADDR3 = 1;
15     ADDR2 = 1;
16     ADDR1 = 1;
17     ADDR0 = 0;
18     while(1)
19     {
20         P0 = ~(0x01 << cnt);
21         for(i=0;i<30000;i++);
22         cnt++;
23         if(cnt>=8)
24         {
25             cnt =0;
26         }
27     }
28 }
29 }
```

作业

■ 操作练习：

1、将流水灯左移理解透彻后，独立完成流水灯右移操作。